Bases de Dados

Window Functions
CTE - Common Table Expressions

Window functions

Definição e tipos de funções

Executam um cálculo sobre um conjunto de registos da tabela, que estão de alguma forma relacionados com o registo atual. A cláusula OVER é obrigatória.

- Duas cláusulas opcionais:
  - PARTITION BY – utilizada para organizar os conjuntos de registos;
  - ORDER BY – utilizada para ordenar os conjuntos de resultados.
  - FRAME – um subconjunto dos registos especificados pela partição.

Existem dois tipos de funções. De agregação e analitícas.

- De agregação
  - SUM, COUNT, AVG, MIN, MAX, etc. Devolvem por norma um valor escalar.
- Analíticas – dividem-se em dois grupos. RANKING e VALOR:
  - RANKING – ROW_NUMBER, RANK, DENSE_RANK, etc.
  - VALOR – FIRST_VALUE, LAST_VALUE, etc.

```
SELECT ... FUNC-NAME(...) OVER (...)
FROM tableName
```

OVER (…)

- OVER(), estabelece o cálculo da função pelo conjunto total de registos
- OVER (PARTITION BY …), estabelece qual os atributos pelos quais se agrupa a informação;
- OVER (PARTITION BY … ORDER BY …), permite estabelecer ordenação específica para a partição.

Restrições

- Não podem ser utilizadas em UPDATE e DELETE (restrição SQL genérica);
  - É permitido, no entanto, utilizá-las em subconsultas nesses comandos.
- Não é permitido a utilização de DISTINCT em funções de agregação (restrição MySQL);
- Não é permitido o encadeamento de funções – SUM(SUM(…)), por exemplo (restrição MySQL).

Window functions vs Agrupamentos

Executam cálculo de agregação sobre diversos registos que estão relacionados com um único valor.

As funções de agregação funcionam como as utilizadas em agrupamentos, mas a informação não é agrupada como na cláusula GROUP BY.

Agrupamento (GROUP BY)

```
SELECT a.titulo,SUM(duracao)
FROM album a INNER JOIN faixas b ON
album_id=album
GROUP BY a.album_id
```

titulo	sum(duracao)
Freak of Nature	2838
Foley Room	3033
Sweet Jones	2697
Ring Ring	3447
Waterloo	3386
ABBA	2888
Arrival	3279
The Album	2923
Voulez-Vous	4154
Super Trouper	3306

Window function

```
SELECT a.titulo,
SUM(duracao) OVER(PARTITION BY album)
as dur_total
FROM album a INNER JOIN faixas b ON
album_id=album
```

titulo	dur_total
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Freak of Nature	2838
Foley Room	3033
Foley Room	3033
Foley Room	3033

Window functions

PARTITION BY

Window function – COUNT()

- `select a.titulo, count(faixa_id) over() as nfaixas from album a inner join faixas b on album_id=album;`
- `select a.titulo, count(faixa_id) over(PARTITION BY album) as nfaixas from album a inner join faixas b on album_id=album;`

titulo	nfaixas
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Freak of Nature	13024
Ring Ring	13024
Ring Ring	13024
Ring Ring	13024
...	...

titulo	nfaixas
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Freak of Nature	12
Ring Ring	19
Ring Ring	19
Ring Ring	19
...	...

Funções analíticas – ROW_NUMBER()

ROW_NUMBER

- `SELECT ROW_NUMBER()OVER() as linha, a.titulo,SUM(duracao) FROM album a INNER JOIN faixas b ON a.album_id=b.album GROUP BY a.album_id`

linha	titulo	sum(duracao)
1	Freak of Nature	2838
2	Foley Room	3033
3	Sweet Jones	2697
4	Ring Ring	3447
5	Waterloo	3386
6	ABBA	2888
7	Arrival	3279
8	The Album	2923
9	Voulez-Vous	4154
10	Super Trouper	3306

Window functions

ORDER BY - GRUPO

ROW_NUMBER

```
SELECT ROW_NUMBER()OVER() as linha,
a.titulo,SUM(duracao) FROM album a
INNER JOIN faixas b ON
a.album_id=b.album
GROUP BY a.album_id
ORDER BY SUM(duração) desc
```

linha	titulo	sum(duracao)
738	The Rolling Stone Magazine's 500 Greatest Songs Of All Time	111216
77	Best Jazz 100	23602
799	Self Defence	18361
147	Ambient Lounge 4	17886
158	CSN Box Set	17582
131	Classical Music for People Who Hate Classical Music	16221
367	Lennon	13420
153	24 Carat	11943
484	Ministry of Sound: The Sound of Bassline	11820

ORDER BY - OVER

ROW_NUMBER

```
SELECT ROW_NUMBER()OVER(ORDER BY
SUM(duração)) as linha, a.titulo,SUM(duracao)
FROM album a INNER JOIN faixas b ON
a.album_id=b.album
GROUP BY a.album_id
```

linha	titulo	sum(duracao)
1	The Rolling Stone Magazine's 500 Greatest Songs Of All Time	111216
2	Best Jazz 100	23602
3	Self Defence	18361
4	Ambient Lounge 4	17886
5	CSN Box Set	17582
6	Classical Music for People Who Hate Classical Music	16221
7	Lennon	13420
8	24 Carat	11943
9	Ministry of Sound: The Sound of Bassline	11820

Funções analíticas – RANK()

RANK, DENSE_RANK

```
SELECT titulo,nfaixas,RANK() OVER(ORDER BY nfaixas
DESC) AS class ,DENSE_RANK() OVER(ORDER BY nfaixas
DESC) AS class_densa
FROM (SELECT a.titulo,COUNT(faixa_id) AS nfaixas
      FROM album a INNER JOIN faixas b ON
      a.album_id=b.album
      GROUP BY a.album_id)  tbl
```

titulo	nfaixas	class	class_densa
The Rolling Stone Magazine's 500 Greatest Songs Of...	500	1	1
Best Jazz 100	100	2	2
CSN Box Set	77	3	3
Anthology 1	60	4	4
Lennon	59	5	5
Classical Music for People Who Hate Classical Musi...	58	6	6
Orphans	56	7	7
24 Carat	51	8	8
Ministry of Sound: The Sound of Bassline	51	8	8
Anthology 3	50	10	9
Anthology 2	45	11	10

Funções analíticas – FIRST_VALUE()

FIRST_VALUE,LAST_VALUE

```
SELECT distinct a.titulo as album,
first_value(b.titulo) over(partition by a.album_id
order by duracao) as faixa,
first_value(duracao) over(partition by a.album_id
order by duracao) as duracaomenor
FROM album a INNER JOIN faixas b ON
a.album_id=b.album
WHERE duracao<>0
```

album	faixa	duracaomenor
Freak of Nature	Goodbye	95
Foley Room	At The End Of The Day	192
Sweet Jones	Grazing Maize	74
Ring Ring	Ah vilka tider [bonus track]	153
Waterloo	Waterloo (French version) [bonus track]	162
ABBA	Man in the middle	183
Arrival	Dum dum diddle	175
The Album	Hole in your soul	221

Window functions

FRAME

Cláusula:unidades e extensão

- Unidades:ROWS|RANGE
- Extensão:
  - FRAME_START|FRAME_BETWEEN
  - FRAME_BETWEEN:
    - BETWEEN FRAME_START and FRAME_END

FRAME_START e FRAME_END

- CURRENT ROW – registo corrente
- UNBOUND PRECEDING – primeiro registo da partição
- UNBOUND FOLLOWING – último registo da partição

FRAME - exemplos

Com e sem ORDER BY

- Com ORDER BY
  - RANGE BETWEEN UNBOUND PRECEDING AND CURRENT ROW
  - Todos os registos entre o 1º e o corrente;
  - Equivalente a ser declado apenas o ORDER BY. Não necessita de ser declarado
- Sem ORDER BY
  - RANGE BETWEEN UNBOUND PRECEDING AND CURRENT ROW
  - Todos os registos da partição. Não necessita de ser declarado.

FRAME_START e FRAME_END

- CURRENT ROW – registo corrente
- UNBOUND PRECEDING – primeiro registo da partição
- UNBOUND FOLLOWING – último registo da partição

Window functions

Exemplo – lista ordenada, em cada álbum, do tempo de início das faixas.

ORDER BY – sem declaração de frame

```
SELECT b.faixa_numero as faixa,b.titulo,
SEC_TO_TIME(SUM(duracao) over (partition by
a.album_id order by b.faixa_numero)-duracao) as
start
FROM album a INNER JOIN faixas b ON
a.album_id=b.album WHERE a.titulo LIKE 'Abbey Road';
```

faixa	titulo	start
1	Come Together	00:00:00
2	Something	00:04:19
3	Maxwell's Silver Hammer	00:07:21
4	Oh! Darling	00:10:48
5	Octopus's Garden	00:14:15
6	I Want You (She's So Heavy)	00:17:05
7	Here Comes the Sun	00:24:52
8	Because	00:27:57
9	You Never Give Me Your Money	00:30:42
10	Sun King	00:34:45
11	Mean Mr. Mustard	00:37:11
12	Polythene Pam	00:38:17
13	She Came In Through the Bathroom Window	00:39:29
14	Golden Slumbers	00:41:27
15	Carry That Weight	00:42:58
16	The End	00:44:34
17	Her Majesty	00:46:55

Exemplo – lista ordenada, em cada álbum, do tempo de início das faixas.

ORDER BY – com declaração de frame

```
SELECT b.faixa_numero as faixa,b.titulo,
SEC_TO_TIME(SUM(duracao) over (partition by
a.album_id order by b.faixa_numero range between
unbounded preceding and current row)-duracao) as
start
FROM album a INNER JOIN faixas b ON
a.album_id=b.album WHERE a.titulo LIKE 'Abbey Road';
```

faixa	titulo	start
1	Come Together	00:00:00
2	Something	00:04:19
3	Maxwell's Silver Hammer	00:07:21
4	Oh! Darling	00:10:48
5	Octopus's Garden	00:14:15
6	I Want You (She's So Heavy)	00:17:05
7	Here Comes the Sun	00:24:52
8	Because	00:27:57
9	You Never Give Me Your Money	00:30:42
10	Sun King	00:34:45
11	Mean Mr. Mustard	00:37:11
12	Polythene Pam	00:38:17
13	She Came In Through the Bathroom Window	00:39:29
14	Golden Slumbers	00:41:27
15	Carry That Weight	00:42:58
16	The End	00:44:34
17	Her Majesty	00:46:55

Window functions

Exemplo – tempo que falta para o final do álbum quando se inicia cada música.

ORDER BY – com declaração de frame

```
SELECT b.faixa_numero as faixa,b.titulo,
SEC_TO_TIME(SUM(duracao) over (partition by
a.album_id order by b.faixa_numero range between
current row and unbounded following)) as fim
FROM album a INNER JOIN faixas b ON
a.album_id=b.album WHERE a.titulo LIKE 'Abbey Road';
```

faixa	titulo	fim
1	Come Together	00:47:21
2	Something	00:43:02
3	Maxwell's Silver Hammer	00:40:00
4	Oh! Darling	00:36:33
5	Octopus's Garden	00:33:06
6	I Want You (She's So Heavy)	00:30:16
7	Here Comes the Sun	00:22:29
8	Because	00:19:24
9	You Never Give Me Your Money	00:16:39
10	Sun King	00:12:36
11	Mean Mr. Mustard	00:10:10
12	Polythene Pam	00:09:04
13	She Came In Through the Bathroom Window	00:07:52
14	Golden Slumbers	00:05:54
15	Carry That Weight	00:04:23
16	The End	00:02:47
17	Her Majesty	00:00:26

Exemplo – lista ordenada, em cada álbum, do tempo de início das faixas.

Declaração de nome para a “janela”

```
SELECT b.faixa_numero as faixa,b.titulo,
SEC_TO_TIME(SUM(duracao) over j-duracao) as start
FROM album a INNER JOIN faixas b ON
a.album_id=b.album WHERE a.titulo LIKE 'Abbey Road’
window j as (partition by a.album_id order by
b.faixa_numero );
```

faixa	titulo	start
1	Come Together	00:00:00
2	Something	00:04:19
3	Maxwell's Silver Hammer	00:07:21
4	Oh! Darling	00:10:48
5	Octopus's Garden	00:14:15
6	I Want You (She's So Heavy)	00:17:05
7	Here Comes the Sun	00:24:52
8	Because	00:27:57
9	You Never Give Me Your Money	00:30:42
10	Sun King	00:34:45
11	Mean Mr. Mustard	00:37:11
12	Polythene Pam	00:38:17
13	She Came In Through the Bathroom Window	00:39:29
14	Golden Slumbers	00:41:27
15	Carry That Weight	00:42:58
16	The End	00:44:34
17	Her Majesty	00:46:55

Common Table Expression - CTE

Introdução

Uma CTE (expressão de tabela comum) é um resultado temporário, referenciado com um nome

- Existem durante a duração do “query” em que estão inseridas.;
- Podem ser encaradas como tabelas derivadas (subqueries na cláusula FROM ou UPDATE) mas são bastante mais poderosas.

Caracteristícas

- Permitem referências a si próprias;
- Podem ser “utilizadas” múltiplas vezes na mesma consulta;
- Permitem agrupamento de colunas;
- Têm um comportamento similar às vistas.

Utilização

Sintaxe

```
WITH <cte_name> AS (
  <cte_body>
)
  <cte_query>
```

Cláusulas

- **`WITH <cte_name> AS`** – identificam que vai ser declarada uma CTE com um determinado nome.
- **`<cte_body>`** - é uma declaração de SELECT que produz um resultado. O corpo da CTE é encapsulada entre parenteses.
- **`<cte_query>`** - declaração que utiliza a referência ao nome da CTE.

Common Table Expression - CTE

Exemplo

Genérico

```
SELECT <atributos>
FROM <cte_name>
[WHERE ...]
```

BD musica

```
with musicas_longas as (
    select * from faixas
    where duracao>240)
 select* from musicas_longas
 where titulo like 'I love%'
 order by titulo;
```

Vantagens

Intrínsecas

- Temporárias – permitem definir fontes de dados que não obrigam a manutenção ou cuidados de utilização especiais.
- Reutilização – no mesmo comando podem ser utilizadas diversas vezes sem que a complexidade do comando aumente.

Outras

- Recursividade – uma CTE pode chamar-se a si própria;
- Permissões – não são necessárias permissões adicionais para criar CTEs.
- Clareza – a escrita de comandos recorrendo a subconsultas (muitas vezes com vários níveis de encadeamento) torna-os muitas vezes opacos e leitura complexa.

Common Table Expression - CTE

Outros exemplos – não recursivos

Quanto minutos de música existem, para cada artista, entre os anos de 1970 e 1979?

```
select a.nome,b.ano_album,sum(c.duracao)
from artista a inner join album b on
a.artista_id=b.artista inner join faixas c on
b.album_id=c.album
where b.ano_album>=1970 and b.ano_album<=1979
group by a.artista_id,b.ano_album
order by a.nome,b.ano_album
```

BD musica

```
with musicas_longas as (
    select * from faixas
    where duracao>240)
 select* from musicas_longas
 where titulo like 'I love%'
 order by titulo;
```

Vantagens

Intrínsecas

- Temporárias – permitem definir fontes de dados que não obrigam a manutenção ou cuidados de utilização especiais.
- Reutilização – no mesmo comando podem ser utilizadas diversas vezes sem que a complexidade do comando aumente.

Outras

- Recursividade – uma CTE pode chamar-se a si própria;
- Permissões – não são necessárias permissões adicionais para criar CTEs.
- Clareza – a escrita de comandos recorrendo a subconsultas (muitas vezes com vários níveis de encadeamento) torna-os muitas vezes opacos e leitura complexa.

CTE– Não recursivas e recursivas

Exemplo (não recursivas)

- Quanto minutos de música existem, para cada artista, entre os anos de 1973 e 1975?

```
select a.nome,b.ano_album,sum(c.duracao)
from artista a inner join album b on a.artista_id=b.artista
               inner join faixas c on b.album_id=c.album
where b.ano_album>=1973 and b.ano_album<=1975
group by a.artista_id,b.ano_album
order by a.nome,b.ano_album
```

- Usando CTEs o resultado pode ser fornecido de forma mais interessante.

```
with dist_ano as (
select a.nome,b.ano_album,sum(c.duracao) as dur
from artista a inner join album b on a.artista_id=b.artista
inner join faixas c on b.album_id=c.album
where b.ano_album>=1973 and b.ano_album<=1975
group by a.artista_id,b.ano_album
order by a.nome,b.ano_album),
artistas as (select distinct nome from dist_ano)

select A.nome,A.ano_album as Ano,A.dur as
Minutos,B.ano_album as Ano,B.dur as Minutos,C.ano_album as
Ano,C.dur as Minutos
from artistas left join dist_ano A on artistas.nome=A.nome
left join dist_ano B on artistas.nome= B.nome left join
dist_ano C on artistas.nome=C.nome
where B.ano_album=A.ano_album+1 and
C.ano_album=B.ano_album+1
```

Exemplo (recursivas)

```
CREATE TABLE animais (
  id serial primary key,
  name VARCHAR(100) NOT NULL,
  father BIGINT(20),
  mother BIGINT(20)
);
```

- A sintaxe é ligeiramente distinta:

```
WITH RECURSIVE <cte_name> AS (
  <anchor>
  UNION [ALL]
  <recursive>)
```

```
WITH RECURSIVE TotalSum AS (
  SELECT
    0 AS Count,
    0 AS Total
  UNION ALL
  SELECT
    Count + 1,
    Total + Count+1
  FROM TotalSum
  WHERE Count < 100
)
SELECT * FROM TotalSum;
```

CTE– Não recursivas e recursivas

Exemplo (recursivas)

- Conhecer os pais do Farrusco?

```
select * from animais where id=8;
```

- Não muito bom. Melhorando...

```
select farrusco.id,farrusco.nome,animais.id,animais.nome
from animais farrusco inner join animais on
animais.id=farrusco.pai or animais.id=farrusco.mae
where farrusco.id=8;
```

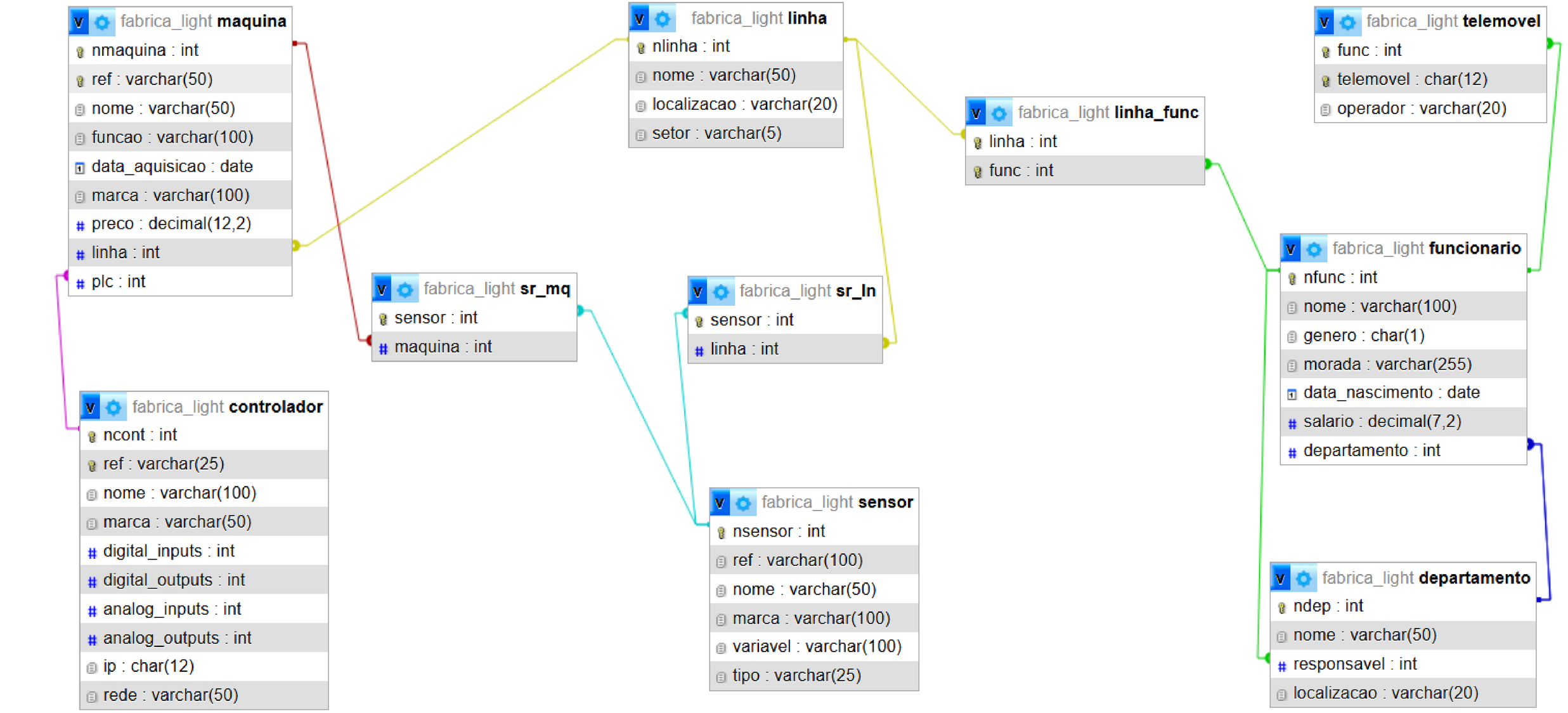
- Melhor, Mas e os avós?.

```
with recursive farrusco as (
    select * from animais
    where id=8
  UNION
    Select animais.*
    from animais,farrusco
    where animais.id=farrusco.pai or
    animais.id=farrusco.mae
)
select * from farrusco;
```

Bases de Dados

Exemplos de Window Functions e CTEs
Base de Dados **fabrica**

Base de Dados fabrica



sola

Window functions

Questão 1 (Função Agregada: AVG)

Liste o nome, o salário e o departamento de cada funcionário, adicionando uma coluna que mostre a média salarial do seu respetivo departamento.

```
SELECT
    nome,salario,departamento,
    AVG(salario) OVER (PARTITION BY departamento) AS
media_salarial_dep
FROM
    funcionário
ORDER BY nome;
```

Explicação

- A função AVG(salario) é usada como função de janela através da cláusula OVER.
- O PARTITION BY departamento agrupa os cálculos por departamento, devolvendo a média específica do departamento em cada linha do funcionário, sem colapsar as linhas como faria um GROUP BY tradicional.

Resultado

nome	salario	departamento	media_salarial_dep
Ana Costa	4900.00		54314.555556
Ana Ferreira	2140.00		42117.783333
Ana Martins	2470.00		42117.783333
Ana Oliveira	2338.00		13045.388889
Ana Pereira	3407.00		13045.388889
Ana Rodrigues	1251.00		31270.272727
Ana Santos	2579.00		62344.312500
Ana Santos	5678.00		54314.555556
André Fernandes	5487.00		54314.555556
André Fernandes	3999.00		54314.555556
André Martins	1385.00		31270.272727
André Oliveira	2267.00		42117.783333
André Pereira	2077.00		42117.783333
André Pereira	1772.00		21863.895833
Andreia Almeida	2348.00		21863.895833
Andreia Costa	2405.00		62344.312500
Andreia Fernandes	2921.00		54314.555556
Andreia Ferreira	1503.00		21863.895833
Andreia Martins	5285.00		54314.555556
Andreia Silva	1243.00		31270.272727
Andreia Silva	2201.00		42117.783333
António Fernandes	2112.00		21863.895833
Beatriz Alves	2787.00		54314.555556
Beatriz Costa	3626.00		13045.388889
Beatriz Costa	1905.00		42117.783333
Beatriz Fernandes	2187.00		42117.783333
Beatriz Ferreira	1219.00		31270.272727
Beatriz Oliveira	1635.00		21863.895833
Beatriz Rodrigues	2119.00		42117.783333
Beatriz Rodrigues	1413.00		21863.895833
Beatriz Santos	2035.00		42117.783333
Bruno Almeida	2378.00		21863.895833
Bruno Costa	1335.00		31270.272727
Bruno Fernandes	1199.00		31270.272727
Bruno Fernandes	1882.00		42117.783333
Bruno Martins	2237.00		42117.783333

Window functions

Questão 2 (Função Analítica: RANK)

Liste as máquinas indicando o seu nome, linha e preço. Adicione uma coluna que atribua um ranking às máquinas dentro de cada linha, ordenadas da mais cara para a mais barata.

```
SELECT
    nome,linha,preco,
RANK() OVER (PARTITION BY linha ORDER BY preco DESC)
AS ranking_preco_linha
FROM
    maquina
WHERE
    linha IS NOT NULL;
```

Explicação

- O RANK() atribui uma posição a cada linha.
- A janela é particionada por linha (reiniciando o ranking em cada linha de produção) e ordenada pelo preco de forma descendente.
- Se duas máquinas tiverem o mesmo preço, recebem o mesmo rank e o rank seguinte é "saltado" (ex: 1, 1, 3).

Resultado

nome	linha	preco	ranking_preco_linha
TOMRA 5C	1	150000.00	1
WECO SORTER S80	1	70000.00	2
SortMaster 100	1	40000.00	3
SortMaster 100	1	40000.00	3
D2200 S	1	12000.00	5
D2200 S	1	12000.00	5
D2200 S	1	12000.00	5
D2200 S	1	12000.00	5
LiftMate 1500	1	3000.00	9
LiftMate 1500	1	3000.00	9
GEWA 3800V PLUS	2	192000.00	1
TOMRA 5A	2	150000.00	2
KW Disinfector	2	60000.00	3
D2200 S	2	12000.00	4
D2200 S	2	12000.00	4
D2200 S	2	12000.00	4
Natrix	2	7000.00	7
Larox PF Pressure Filte	3	280000.00	1
Alfa Laval AS-H Plate Press	3	100000.00	2
Tomato Crusher TC	3	28000.00	3
Tomato Crusher TC	3	23000.00	4
D2200 S	3	12000.00	5
D2200 S	3	12000.00	5
Comitrol® Processor	3	10000.00	7
Comitrol® Processor	3	9000.00	8
Natrix	3	7000.00	9
Natrix	3	7000.00	9
Tetra Alfast 210	4	50500.00	1
Tetra Alfast 210	4	50500.00	1

Window functions

Questão 3 (Função Analítica: LAG)

Liste as máquinas ordenadas pela data de aquisição. Mostre o nome, data de aquisição, preço, e calcule a diferença de preço entre a máquina atual e a máquina imediatamente anterior a ser comprada (em toda a fábrica).

```
SELECT
    nome,
    data_aquisicao,
    preco,
    preco - LAG(preco) OVER (ORDER BY data_aquisicao)
AS diferenca_preco_anterior
FROM
    maquina;
```

Explicação

- A função LAG(preco) acede ao valor da coluna preco da linha anterior, baseando-se na ordenação da data_aquisicao.
- Subtraindo este valor ao preço atual, obtemos a diferença. Na primeira linha o valor do LAG será NULL.

Resultado

nome	data_aquisicao	preco	diferenca_preco_anterior
EZSpan	02/06/2018	3500.00	NULL
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
EZSpan	02/06/2018	3500.00	0.00
SortMaster 100	02/06/2018	40000.00	36500.00
EZSpan	02/06/2018	3500.00	-36500.00
SortMaster 100	03/07/2018	40000.00	36500.00
TOMRA 5C	03/08/2018	150000.00	110000.00
MP150	03/09/2018	122000.00	-28000.00
MP150	03/09/2018	122000.00	0.00
MP150	03/09/2018	122000.00	0.00
WECO SORTER S80	03/09/2018	70000.00	-52000.00
Canmatic	03/09/2018	200000.00	130000.00
Opera Roll-Fed Labeler	03/09/2018	150000.00	-50000.00
Opera Roll-Fed Labeler	03/09/2018	150000.00	0.00
Krones Contipure Aseptic Line	03/09/2018	400000.00	250000.00
Krones Contipure Aseptic Line	03/09/2018	400000.00	0.00
Krones Pasteurizer	03/09/2018	750000.00	350000.00
Canmatic	03/09/2018	200000.00	-550000.00
Natrix	03/09/2018	7000.00	-193000.00
Natrix	03/09/2018	7000.00	0.00
Natrix	03/09/2018	7000.00	0.00
MP150	03/09/2018	122000.00	115000.00
TOMRA 5A	04/10/2018	150000.00	28000.00
GEWA 3800V PLUS	04/11/2018	192000.00	42000.00

Window functions

Questão 4 (Função Agregada: SUM Acumulado)

Crie um relatório que mostre o nome da máquina, a data de aquisição, o preço e o **custo acumulado** das máquinas ao longo do tempo (soma cumulativa dos preços, ordenada pela data de aquisição).

```
SELECT
    nome,data_aquisicao,preco,
    SUM(preco) OVER (ORDER BY data_aquisicao) AS
custo_acumulado
FROM
    maquina;
```

Explicação

- Quando utilizamos uma função agregada como SUM() com um ORDER BY dentro da cláusula OVER() (e sem PARTITION BY), o MySQL assume por defeito um "running total" (soma cumulativa) desde a primeira linha até à linha atual da partição.
- Neste caso a partição é dada pelas diferentes datas.

Resultado

nome	data_aquisicao	preco	custo_acumulado
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
EZSpan		02/06/20183500.00	71500.00
SortMaster 100		02/06/201840000.00	71500.00
EZSpan		02/06/20183500.00	71500.00
SortMaster 100		03/07/201840000.00	111500.00
TOMRA 5C		03/08/2018150000.00	261500.00
MP150		03/09/2018122000.00	3090500.00
MP150		03/09/2018122000.00	3090500.00
MP150		03/09/2018122000.00	3090500.00
WECO SORTER S80		03/09/201870000.00	3090500.00
Canmatic		03/09/2018200000.00	3090500.00
Opera Roll-Fed Labeler		03/09/2018150000.00	3090500.00
Opera Roll-Fed Labeler		03/09/2018150000.00	3090500.00
Krones Contipure Aseptic Line		03/09/2018400000.00	3090500.00
Krones Contipure Aseptic Line		03/09/2018400000.00	3090500.00
Krones Pasteurizer		03/09/2018750000.00	3090500.00
Canmatic		03/09/2018200000.00	3090500.00
Natrix		03/09/20187000.00	3090500.00

Window functions

Questão 5 (Função Analítica: ROW_NUMBER com Subconsulta)

Utilizando window functions, liste apenas os **2 funcionários mais bem pagos** de cada departamento. (Mostre nome, departamento e salário).

```
SELECT nome, departamento, salario
FROM (SELECT
        nome,departamento,salario,
        ROW_NUMBER() OVER (PARTITION BY departamento
ORDER BY salario DESC) AS rn
    FROM funcionário AS subquery
WHERE rn <= 2;
```

Explicação

- As window functions não podem ser usadas diretamente na cláusula WHERE.
- Não utilizando CTEs, criamos uma subconsulta no FROM. O ROW_NUMBER() atribui um número sequencial único (1, 2, 3...) dentro de cada departamento baseado no maior salário. A consulta externa filtra apenas aqueles cuja posição é 1 ou 2.

Resultado

nome	departamento	posicao	salario
Mariana Almeida	1	1	3678.00
Beatriz Costa	1	2	3626.00
Sofia Pereira	2	1	2524.00
Manuel Santos	2	2	2490.00
João Pereira	3	1	1467.00
Margarida Santos	3	2	1462.00
Filipa Rodrigues	4	1	2680.00
Carlos Almeida	4	2	2655.00
Mariana Martins	5	1	6134.00
Pedro Silva	5	2	6056.00
João Ferreira	6	1	2610.00
Ana Santos	6	2	2579.00

Window functions

Questão 6 (Função Analítica: FIRST_VALUE)

Liste o nome e salário de cada funcionário, e inclua uma coluna que mostre o salário mais alto do departamento em que ele trabalha, usando a função FIRST_VALUE.

```
SELECT
    nome,
    salario,
    departamento,
    FIRST_VALUE(salario) OVER (PARTITION BY
departamento ORDER BY salario DESC) AS
salario_maximo_dep
FROM funcionario;
```

Explicação

- A função FIRST_VALUE() devolve o primeiro valor de um conjunto ordenado.
- Ao particionar por departamento e ordenar pelo salario de forma descendente, o primeiro valor da janela é garantidamente o salário mais alto daquele departamento.

Resultado

nome	salario	departamento	salario_maximo_dep
Ana Costa	4900.00	5	6134.00
Ana Ferreira	2140.00	4	2680.00
Ana Martins	2470.00	4	2680.00
Ana Oliveira	2338.00	1	3678.00
Ana Pereira	3407.00	1	3678.00
Ana Rodrigues	1251.00	3	1467.00
Ana Santos	2579.00	6	2610.00
Ana Santos	5678.00	5	6134.00
André Fernandes	3999.00	5	6134.00
André Fernandes	5487.00	5	6134.00
André Martins	1385.00	3	1467.00
André Oliveira	2267.00	4	2680.00
André Pereira	2077.00	4	2680.00
André Pereira	1772.00	2	2524.00
Andreia Almeida	2348.00	2	2524.00
Andreia Costa	2405.00	6	2610.00
Andreia Fernandes	2921.00	5	6134.00
Andreia Ferreira	1503.00	2	2524.00
Andreia Martins	5285.00	5	6134.00
Andreia Silva	2201.00	4	2680.00
Andreia Silva	1243.00	3	1467.00
António Fernandes	2112.00	2	2524.00
Beatriz Alves	2787.00	5	6134.00
Beatriz Costa	3626.00	1	3678.00

Window functions

Questão 7 (Função Agregada: COUNT)

Para cada sensor da fábrica, liste o seu nome, tipo, marca, e mostre o total de sensores que a fábrica possui dessa mesma marca.

```
SELECT
    nome,tipo,marca,
    COUNT(nsensor) OVER (PARTITION BY marca) AS
total_sensores_marca
FROM
    sensor
ORDER BY
    nome;
```

Explicação

- Usamos o COUNT() sobre a chave primária (nsensor).
- O PARTITION BY marca indica que o contador deve contar as linhas agrupadas pela marca.
- Ao invés de resumir a listagem apenas às marcas (como o GROUP BY), a informação original de cada sensor mantém-se visível.

Resultado

nome	tipo	marca	total_sensores_marca
5103	Analógico	RM Young	3
05103-45	Analógico	RM Young	3
080D100	Analógico	PCB Piezotronics	7
1250	Analógico	Tedea-Huntleigh	1
333D01	Analógico	PCB Piezotronics	7
352A74	Analógico	PCB Piezotronics	7
356A12	Digital	PCB Piezotronics	7
356A21	Analógico	PCB Piezotronics	7
378B02	Analógico	PCB Piezotronics	7
378B04	Analógico	PCB Piezotronics	7
ACS712	Digital	Allegro Microsystems	6
ACS712	Analógico	Allegro Microsystems	6
ACS714	Analógico	Allegro Microsystems	6
ACS758	Digital	Allegro Microsystems	6
ACS758	Analógico	Allegro Microsystems	6
ACS758LCB	Analógico	Allegro Microsystems	6
ADIS16209	Digital	Analog Devices	4
ADS1115	Digital	Texas Instruments	25
ADXL335	Digital	Analog Devices	4
ADXL345	Digital	Analog Devices	4
BMP280	Digital	Bosch Sensortec	14
BMP280	Digital	Bosch Sensortec	14
BMP280	Digital	Bosch Sensortec	14
BMP280	Digital	Bosch Sensortec	14

Window functions

Questão 8 (Função Analítica: NTILE)

Divida todos os funcionários da fábrica em 4 grupos (quartis) com base nos seus salários, do maior para o menor, para análise de equidade salarial. Mostre nome, salário e o número do quartil (1 a 4).

```
SELECT
    nome,salario,
    NTILE(4) OVER (ORDER BY salario DESC) AS
quartil_salarial
FROM
    funcionário
ORDER BY
    nome;
```

Explicação

- A função NTILE(N) tenta dividir as linhas do resultado em N grupos o mais iguais possível.
- Com a ordenação pelo salário em ordem decrescente, o grupo 1 (quartil 1) terá os trabalhadores mais bem pagos, e o grupo 4 terá os que auferem salários mais baixos.

Resultado

nome	salario	quartil_salarial
Ana Costa	4900.00	1
Ana Ferreira	2140.00	3
Ana Martins	2470.00	2
Ana Oliveira	2338.00	2
Ana Pereira	3407.00	1
Ana Rodrigues	1251.00	4
Ana Santos	5678.00	1
Ana Santos	2579.00	1
André Fernandes	5487.00	1
André Fernandes	3999.00	1
André Martins	1385.00	4
André Oliveira	2267.00	2
André Pereira	2077.00	3
André Pereira	1772.00	4
Andreia Almeida	2348.00	2
Andreia Costa	2405.00	2
Andreia Fernandes	2921.00	1
Andreia Ferreira	1503.00	4
Andreia Martins	5285.00	1
Andreia Silva	1243.00	4
Andreia Silva	2201.00	2
António Fernandes	2112.00	3
Beatriz Alves	2787.00	1
Beatriz Costa	1905.00	3

Window functions

Questão 9 (Funções Analíticas e de Sistema: LEAD e DATEDIFF)

Para cada funcionário de um determinado departamento, liste o nome e a data de nascimento. Mostre a diferença em dias entre a data de nascimento do funcionário atual e a do funcionário imediatamente mais novo do mesmo departamento.

```
SELECT
    nome,departamento,data_nascimento,
DATEDIFF(LEAD(data_nascimento) OVER (PARTITION BY
departamento ORDER BY data_nascimento ASC),
data_nascimento
    ) AS diferenca_dias_prox_novo
FROM funcionario
ORDER BY nome;
```

Explicação

- O LEAD(data_nascimento) "olha para a frente" na janela, capturando o valor da linha seguinte.
- Como ordenámos a data de nascimento por ordem crescente (do mais velho para o mais novo), a linha seguinte pertence ao funcionário imediatamente mais novo.
- A função padrão DATEDIFF calcula a diferença em dias entre estas duas datas.

Resultado

nome	departamento	data_nascimento	diferenca_dias_prox_novo
Ana Costa	5	08/11/1979	259
Ana Ferreira	4	24/07/1991	269
Ana Martins	4	29/09/1985	185
Ana Oliveira	1	03/08/1970	1335
Ana Pereira	1	03/11/1992	203
Ana Rodrigues	3	22/06/1983	352
Ana Santos	5	28/02/1981	849
Ana Santos	6	07/11/1979	1117
André Fernandes	5	29/01/1971	992
André Fernandes	5	10/03/1977	1
André Martins	3	30/03/1988	336
André Oliveira	4	29/03/1985	184
André Pereira	2	09/01/1988	33
André Pereira	4	15/09/1990	228
Andreia Almeida	2	10/09/1970	20
Andreia Costa	6	20/09/1976	139
Andreia Fernandes	5	03/12/1976	49
Andreia Ferreira	2	12/06/1983	220
Andreia Martins	5	13/05/1993	0
Andreia Silva	3	03/06/1978	183
Andreia Silva	4	09/08/1982	302
António Fernandes	2	17/07/1991	292
Beatriz Alves	5	25/03/1984	177
Beatriz Costa	1	29/05/1982	318

Window functions

Questão 10 (Função Agregada: MAX vs Valor Atual)

Calcule a percentagem de digital inputs de cada controlador em relação ao máximo de digital inputs encontrados num controlador que pertença à mesma rede. Liste o nome do controlador, rede, digital_inputs, e a percentagem calculada (como valor entre 0 e 100).

```
SELECT
    nome,rede,digital_inputs,
    (digital_inputs / MAX(digital_inputs) OVER
(PARTITION BY rede)) * 100 AS percentagem_uso_rede
FROM
    controlador
WHERE
    rede IS NOT NULL;
```

Explicação

- Esta query combina operações aritméticas com window functions na mesma expressão matemática.
- Primeiro usamos o MAX(digital_inputs) OVER (PARTITION BY rede) para encontrar o valor mais alto daquela rede em específico, e a seguir dividimos o valor atual (digital_inputs daquela linha) pelo valor da janela, multiplicando por 100.

Resultado

nome	rede	digital_inputs	percentagem_uso_rede
S7-414-3 PN/DP		512	100.0000
S7-1214C	LANcaprina	64	25.0000
S7-1214C	LANcaprina	64	25.0000
S7-1214C	LANcaprina	64	25.0000
s7-1517PN	LANcaprina	256	100.0000
s7-1517PN	LANcaprina	256	100.0000
S7-1214C	LANdústria do Riso	64	12.5000
S7-1214C	LANdústria do Riso	64	12.5000
S7-1214C	LANdústria do Riso	64	12.5000
s7-1517PN	LANdústria do Riso	512	100.0000
V350-35-R34	LANterna Mágica	32	25.0000
CP1H-XA40DT-D	LANterna Mágica	128	100.0000
CP1H-XA40DT-D	LANterna Mágica	128	100.0000
CP1H-XA40DT-D	LANterna Mágica	128	100.0000

CTEs

Questão 1 (CTE Simples de Agregação)

Apresente o nome de cada linha de produção e o custo total das máquinas associadas a essa mesma linha. Ordene o resultado da linha com maior investimento para a de menor.

```
WITH CustoMaquinasPorLinha AS (
    SELECT
        linha,
        SUM(preco) AS custo_total
    FROM
        maquina
    WHERE
        linha IS NOT NULL
    GROUP BY
        linha
)
SELECT
    l.nome,
    c.custo_total
FROM
    linha l
JOIN
    CustoMaquinasPorLinha c ON l.nlinha = c.linha
ORDER BY
    c.custo_total DESC;
```

Resultado

nome	custo_total
Armazenamento e distribuição	7547000.00
Embalagem e rotulagem	1571100.00
Tratamento de resíduos	1300600.00
Pasteurização	750000.00
Concentração por evaporação	507000.00
Descascamento e trituração	488000.00
Refinação e filtração	457400.00
Lavagem e desinfeção	445000.00
Recepção e Selecção	354000.00
Extração	132000.00
Preaquecimento	108000.00

Explicação

- A CTE CustoMaquinasPorLinha calcula primeiro o agrupamento e a soma dos preços por identificador de linha.
- Na consulta principal (o SELECT externo), fazemos um simples JOIN com a tabela linha para ir buscar o nome entendível da linha, mantendo a lógica de agregação separada e fácil de ler.

Questão 2 (CTE com Window Functions para Filtragem)

Apresente o nome de cada linha de produção e o custo total das máquinas associadas a essa mesma linha. Ordene o resultado da linha com maior investimento para a de menor.

```
WITH SalariosDepartamento AS (
    SELECT
        nome,
        salario,
        departamento,
        AVG(salario) OVER (PARTITION BY departamento)
AS media_salarial_dep
    FROM
        funcionario
)
SELECT
    nome,
    salario,
    media_salarial_dep
FROM
    SalariosDepartamento
WHERE
    salario > media_salarial_dep;
```

Resultado

nome	salario	media_salarial_dep
Ana Costa	4900.00	4314.555556
Ana Ferreira	2140.00	2117.783333
Ana Martins	2470.00	2117.783333
Ana Pereira	3407.00	3045.388889
Ana Santos	2579.00	2344.312500
Ana Santos	5678.00	4314.555556
André Fernandes	5487.00	4314.555556
André Martins	1385.00	1270.272727
André Oliveira	2267.00	2117.783333
Andreia Almeida	2348.00	1863.895833
Andreia Costa	2405.00	2344.312500
Andreia Martins	5285.00	4314.555556
Andreia Silva	2201.00	2117.783333
António Fernandes	2112.00	1863.895833
Beatriz Costa	3626.00	3045.388889
Beatriz Fernandes	2187.00	2117.783333

Explicação

- Como as Window Functions (neste caso o AVG() OVER(...)) não podem ser utilizadas diretamente numa cláusula WHERE, a CTE resolve este problema.
- Criamos uma tabela temporária virtual (SalariosDepartamento) que já contém a coluna da média calculada para cada linha. Na consulta externa, podemos agora filtrar facilmente onde o salario é superior a essa média.

CTEs

Questão 3 (Múltiplas CTEs Encadeadas)

Apresente o nome de cada linha de produção e o custo total das máquinas associadas a essa mesma linha. Ordene o resultado da linha com maior investimento para a de menor.

```
WITH MediaPorDepartamento AS (
    SELECT
        departamento,AVG(salario) AS media_salario
    FROM funcionario
    WHERE departamento IS NOT NULL
    GROUP BY departamento
),
MaiorMedia AS (
    SELECT MAX(media_salario) AS valor_maximo
    FROM MediaPorDepartamento
)
SELECT
    d.nome AS nome_departamento,md.media_salario
FROM
    departamento d
JOIN
    MediaPorDepartamento md ON d.ndep =
md.departamento
JOIN
    MaiorMedia mm ON md.media_salario =
mm.valor_maximo;
```

Resultado

nome_departamento	media_salario
Engenharia	4314.555556

Explicação

- Podemos definir várias CTEs separadas por vírgulas.
- A primeira CTE calcula a média de cada departamento.
- A segunda CTE "lê" a primeira CTE para descobrir qual é o valor máximo absoluto entre essas médias.
- A consulta final junta tudo para ligar esse valor máximo ao nome do departamento correspondente na tabela departamento.

CTEs

Questão 4 (CTE para Agregação Condicional)

Liste o nome dos controladores lógicos (PLCs) que têm **mais do que 1 máquina** ligada a eles, mostrando também o total de máquinas que controlam.

```
WITH MaquinasPorPLC AS (
    SELECT
        plc,
        COUNT(nmaquina) AS total_maquinas
    FROM
        maquina
    WHERE
        plc IS NOT NULL
    GROUP BY
        plc
)
SELECT
    c.nome AS nome_controlador,
    m.total_maquinas
FROM
    controlador c
JOIN
    MaquinasPorPLC m ON c.ncont = m.plc
WHERE
    m.total_maquinas > 1;
```

Resultado

nome_controlador	total_maquinas
S7-1214C	3
S7-1214C	3
S7-1214C	4
S7-1214C	2
S7-1214C	4
S7-1214C	3
s7-1517PN	7
s7-1517PN	6
s7-1517PN	6
V350-35-R34	2
CP1H-XA40DT-D	4
CP1H-XA40DT-D	12
CP1H-XA40DT-D	15
S7-414-3 PN/DP	19

Explicação

- Em vez de usar um GROUP BY e um HAVING na mesma consulta, a CTE isola a lógica de contagem e agrupamento de máquinas por PLC.
- A consulta principal apenas liga os IDs aos nomes e aplica o filtro > 1.

CTEs

Questão 5 (Cruzamento de Duas CTEs Diferentes)

Identifique as linhas de produção que são "complexas", ou seja, aquelas que têm em simultâneo pelo menos 2 funcionários alocados e pelo menos 2 máquinas instaladas.

```
WITH TotalMaquinasLinha AS (
    SELECT linha,COUNT(nmaquina) AS qtd_maquinas
    FROM maquina
    GROUP BY linha),
TotalFuncionariosLinha AS (
    SELECT linha, COUNT(func) AS qtd_funcionarios
    FROM linha_func
    GROUP BY linha
)
SELECT
    l.nome AS nome_linha,
    m.qtd_maquinas,
    f.qtd_funcionarios
FROM
    linha l
JOIN
    TotalMaquinasLinha m ON l.nlinha = m.linha
JOIN
    TotalFuncionariosLinha f ON l.nlinha = f.linha
WHERE
    m.qtd_maquinas >= 2
    AND f.qtd_funcionarios >= 2;
```

Resultado

nome_linha	qtd_maquinas	qtd_funcionarios
Recepção e Selecção	10	3
Lavagem e desinfeção	7	3
Descascamento e trituração	10	4
Preaquecimento	4	3
Extração	4	24
Concentração por evaporação	3	3
Refinação e filtração	5	16
Tratamento de resíduos	12	3
Embalagem e rotulagem	15	3
Armazenamento e distribuição	19	22

Explicação

- Esta é uma das situações onde as CTEs se tornam mais importantes. Precisamos de contar dados de duas tabelas diferentes (maquina e linha_func) que partilham a mesma chave (linha). Se fizéssemos JOIN a tudo antes de contar, poderíamos criar duplicações (produto cartesiano). As duas CTEs agregam os dados de forma completamente isolada, e a consulta final junta os resultados agregados de forma correta.

CTEs com recursividade

Questão 1 (Gerar Sequência Numérica para IDs)

A equipa de Recursos Humanos quer pré-alocar os próximos 5 números de identificação (nfunc) para uma nova fornada de funcionários que vai ser contratada, garantindo que os números são consecutivos após o último funcionário registado. Usando uma CTE Recursiva, liste esses 5 próximos IDs disponíveis.

```
WITH RECURSIVE ProximosIDs AS (
-- âncora: Procurar o ID mais alto atual na tabela e somar 1 (o primeiro novo ID)
    SELECT MAX(nfunc) + 1 AS novo_id,1 AS iteracao
    FROM funcionario

    UNION ALL

-- Passo Recursivo: Pegar no resultado anterior e somar 1 ao ID
    SELECT novo_id + 1,iteracao + 1
    FROM
        ProximosIDs
    WHERE
        iteracao < 5
-- Condição de paragem: parar ao fim de 5 iterações
)
SELECT
    novo_id AS ids_reservados
FROM
    ProximosIDs;
```

Resultado

ids_reservados
201
202
203
204
205

Explicação

- Em MySQL, a palavra-chave RECURSIVE é obrigatória quando a CTE chama o seu próprio nome.
- A Âncora (antes do UNION ALL) descobre o número de funcionário mais alto e cria a iteração 1.
- O "Passo Recursivo" (depois do UNION ALL) toma a linha gerada anteriormente, soma 1 ao novo_id e soma 1 à iteracao.
- A cláusula WHERE iteracao < 5 é o nosso travão de segurança para impedir um ciclo infinito, parando assim que gera os 5 números desejados.

CTEs com recursividade

Questão 2 (Gerar Série Contínua de Anos/Datas)

O departamento financeiro pediu um relatório que exige analisar cada ano civil desde a compra da máquina mais antiga até ao ano de compra da máquina mais recente. Acontece que pode haver anos intermédios em que não se compraram máquinas e eles não existiriam num simples SELECT. Crie uma CTE Recursiva que gere uma lista com todos os anos, sem falhas, desde o primeiro até ao último ano de aquisição.

```
WITH RECURSIVE AnosAquisicao AS (
    -- Âncora: Descobrir o primeiro ano (mínimo) e o
último ano (máximo)
    SELECT
        MIN(YEAR(data_aquisicao)) AS ano_atual,
        MAX(YEAR(data_aquisicao)) AS ano_maximo
    FROM maquina
    WHERE data_aquisicao IS NOT NULL

    UNION ALL

    -- Passo Recursivo: Somar 1 ano ao ano atual
gerado na linha anterior
    SELECT ano_atual + 1,ano_maximo
    FROM AnosAquisicao
    WHERE ano_atual < ano_maximo -- Parar quando o ano
gerado atingir o ano máximo
)
SELECT
    ano_atual AS ano_analise_financeira
FROM
    AnosAquisicao;
```

Resultado

ano_analise_financeira
2018
2019
2020
2021

Explicação

- Este é um dos usos mais úteis das CTEs recursivas: preencher “buracos” nos dados.
- A âncora obtém os limites (ex: 2010 a 2024) extraindo apenas o ano da data_aquisicao.
- O Passo Recursivo gera continuamente a linha "anterior + 1" até que o ano_atual empate com o ano_maximo.
- Desta forma, mesmo que a fábrica não tenha comprado nenhuma máquina em 2015, o ano "2015" vai constar na grelha final devolvida pelo SELECT, servindo de base perfeita para ligar (via LEFT JOIN) aos custos reais.